

requirements.txt

The requirements.txt file contains a list of Python packages required for the Hailo examples. To view the file contents:

```
$> cat requirements.txt
```

Expected contents:

numpy < 2.0.0 setproctitle

opencv-python

To install them individually:

```
$> pip install numpy<2.0.0
```

```
$> pip install setproctitle
```

```
$> pip install opencv-python
```

Step 5: Download the necessary Hailo models

```
$> download_resources.sh
```

To install additional Python resources, visit: <https://github.com/hailo-ai/hailo-apps-infra>

This repository provides Hailo Applications Infrastructure for Raspberry Pi 5. Some parts of it are optional for the current code.

```
$> git clone https://github.com/hailo-ai/hailo-apps-infra.git
$> pip install --force-reinstall -v -e .
# (Optional)
```

The 'git clone' command downloads additional Python utilities for Hailo. The last pip command is entirely optional; it may be run or skipped. At this stage, all software components required for the system should be in place. Note that all of these tools fall under the category of open source software.

Begin experimenting with the Hailo-8L accelerator using Python

scripts in the 'basic_pipelines' directory:

Now the process can move on to working with the Hailo-8L accelerator, trying out some sample Python scripts, and eventually building individual measure_distance3.py. Two sample Python scripts are already available in the basic_pipelines subdirectory:

- detection_simple.py

py

- pose_estimation.py

These can accept input from pre-defined sources such as MP4 files, the Raspberry Pi camera, or a USB camera. Example usage:

```
$> python basic_pipelines/detection_simple.py --input rpicam # From Pi Camera
$> python basic_pipelines/pose_estimation.py --input test.mp4 # From video file
$> python basic_pipelines/detection_simple.py --input /dev/video0 # From USB camera
$> python basic_pipelines/detection.py --help # Help for detection.py
```

To use custom models:

```
$> python basic_pipelines/detection.py \
--labels-json resources/barcode-labels.json \
--hef-path resources/yolov8s-hailo8l-barcode.hef \
--input resources/barcode.mp4
```

Distance measurement theory

Distance is estimated by comparing an object's known width with its perceived width in the captured image.

Based on the pinhole camera model, an object farther from the camera appears smaller. If the camera's focal length, the object's real-world width, and the observed image width are known, the distance can be calculated using similar triangles.

Standard widths and a default width are defined at the beginning of the code.

The segmentation model 'yolov5n_seg_h8l_mz.hef' helps identify full objects, even when partially visible. The segmented image is used to estimate the object's width, which is then compared to known dimensions to determine the distance.

```
# Object width dictionary for distance estimation (in metres)
OBJECT_WIDTHS = {
```

```
"person": 0.4, "bicycle": 0.5, "car": 1.8, "motorcycle": 0.8, "bus": 2.5,
```

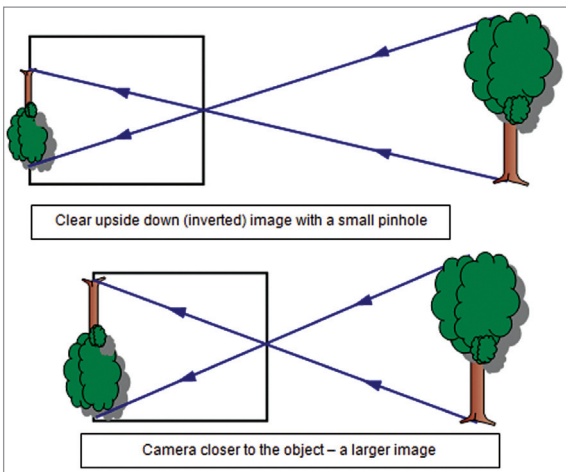
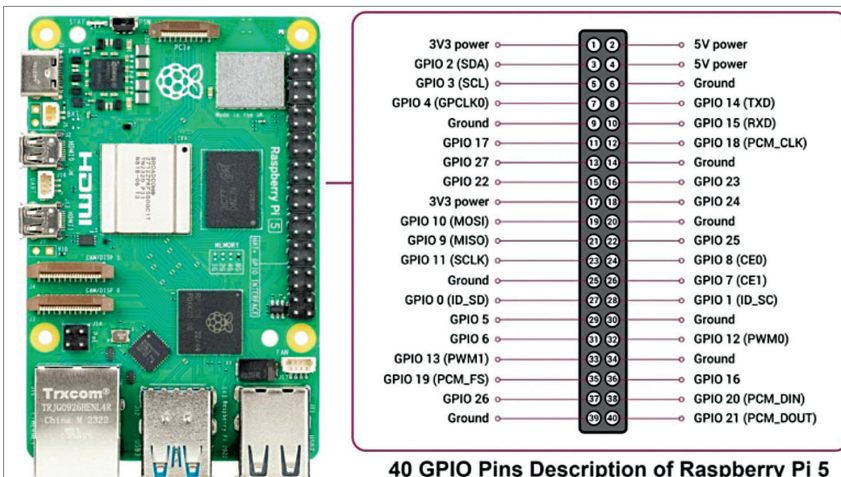


Fig. 3: Distance measure technique



40 GPIO Pins Description of Raspberry Pi 5

Fig. 4: Pin details of the Raspberry Pi